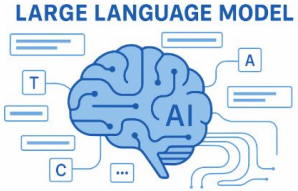
실전 웹서비스개발 - 10주차

인공지능과 웹 서비스 개발

인공지능 기술의 발전

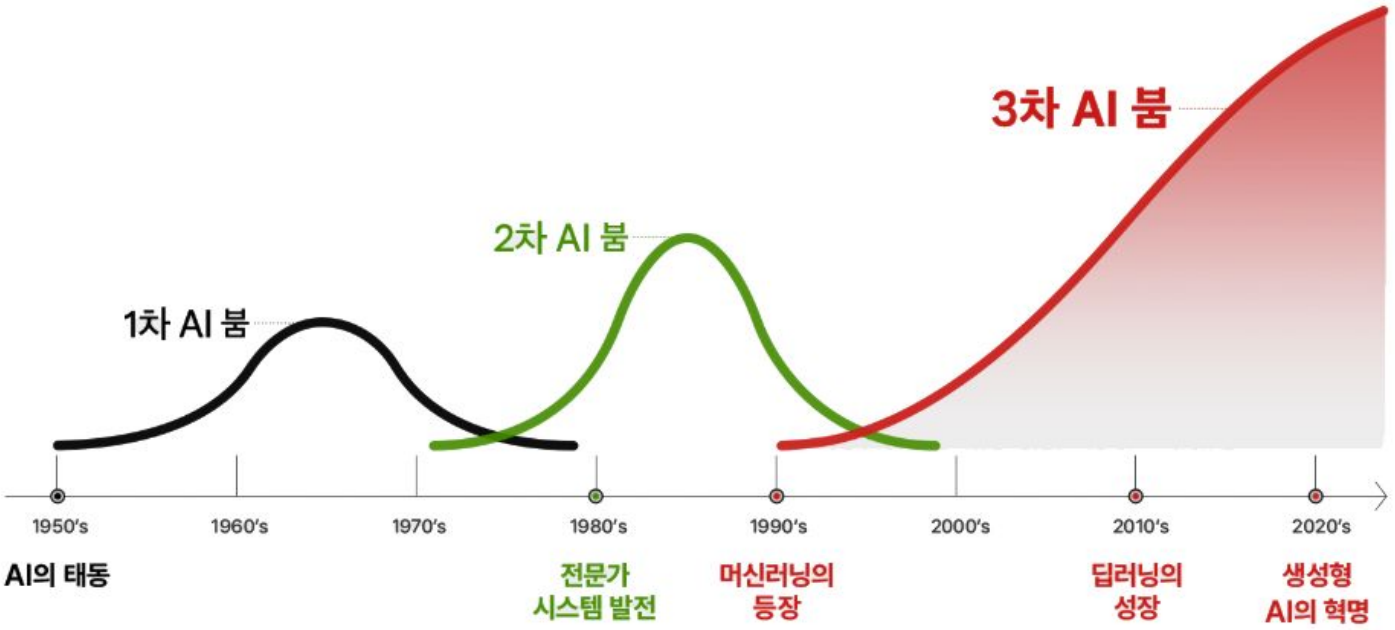


ChatGPT

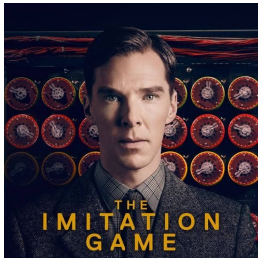
Claude

Gemini

인공지능 기술의 발전


3차 AI 붐
2차 AI 붐
1차 AI 붐
1950's
1960's
1970's
1980's
1990's
2000's
2010's
2020's
AI의 태동
전문가 시스템 발전
머신러닝의 등장
딥러닝의 성장
생성형 AI의 혁명

튜링 테스트



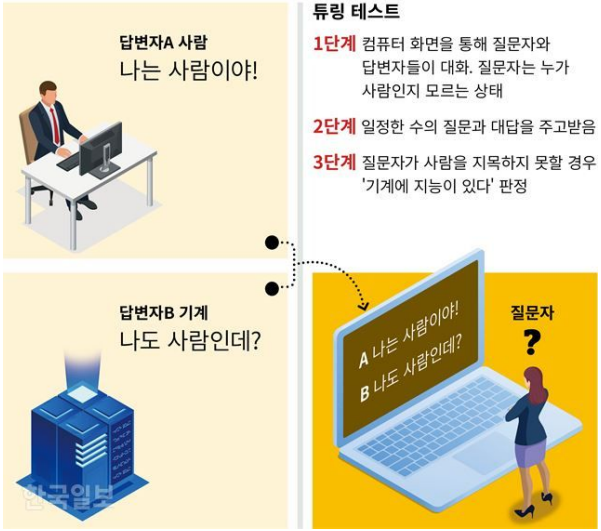
이미테이션 게임 2014

개념

- “기계가 생각할 수 있는가?” — 앨런 튜링 (Alan Turing, 1950)
- 인간과 컴퓨터가 대화를 나눌 때, 사람이 기계와 인간을 구별하지 못한다면 그 컴퓨터는 “지능이 있다”고 판단.

실험 방법

역할	설명
질문자 (심사자)	키보드로 익명 상대에게 질문
피실험자 (기계)	질문에 인간처럼 답변
피실험자 (인간)	동일한 질문에 답변
판정 기준	질문자가 누구(인간/기계)인지 구별하지 못하면 성공



튜링 테스트 실험

의의

- 인공지능 연구의 철학적 출발점
- “사고(thinking)”의 정의를 행동적 관찰로 전환
- 이후 Symbolic AI → Machine Learning → LLM으로 이어지는 기반

퍼셉트론 모델

개념

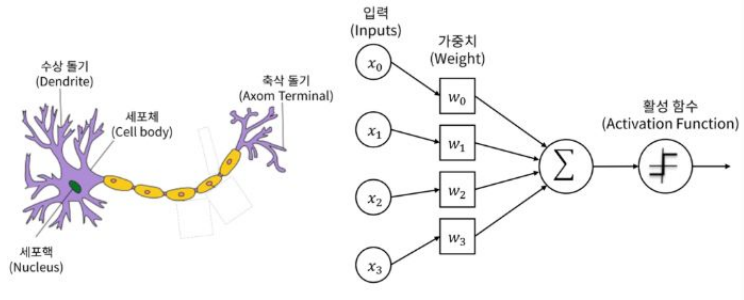
- 프랑크 로젠블라트(Frank Rosenblatt)가 1957년에 제안한 초기 형태의 인공 신경망
- 인간의 뉴런을 모방한 최초의 인공지능 모델

원리

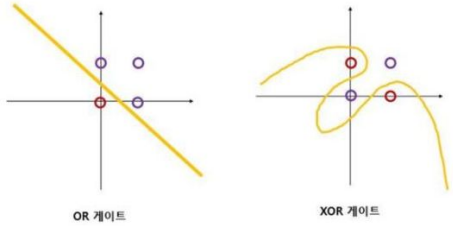
- 입력 신호(Features)에 가중치(Weight)를 곱해 합산한 뒤
- 임계값(Threshold)을 기준으로 출력(Activation)을 결정하는 구조

의의

- 뉴런의 계산 원리를 단순 모델로 구현
- 인공 신경망의 가장 기본 단위를 정의(딥러닝의 시초)
- 1960년대 AI 연구의 핵심 모델
- XOR 문제 해결 불가



퍼셉트론 모델



XOR 문제

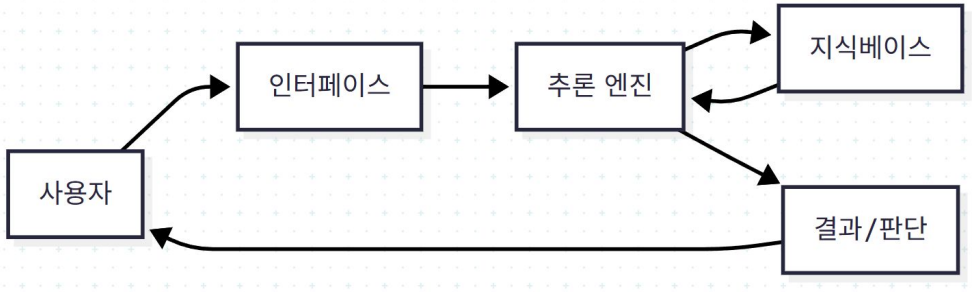
전문가 시스템

개념

- “인간 전문가의 지식을 컴퓨터가 대신 판단하도록 만든 AI 시스템”
- 특정 전문 영역의 규칙과 지식을 지식베이스(knowledge base)에 저장하고 추론엔진(inference engine)을 통해 문제를 해결

원리

- 지식베이스에 전문가의 규칙(If–Then 형태)을 저장
- 추론엔진이 입력된 문제를 분석해 규칙 적용
- 설명 모듈이 판단 근거를 사용자에게 제시



전문가 시스템

의의

- 인간 전문가의 판단을 모방한 지식기반 AI의 시작
- 이후 지식 표현(Knowledge Representation), 추론(Reasoning) 연구로 발전
- 현재의 규칙 기반 챗봇, 의료진단 AI의 전신

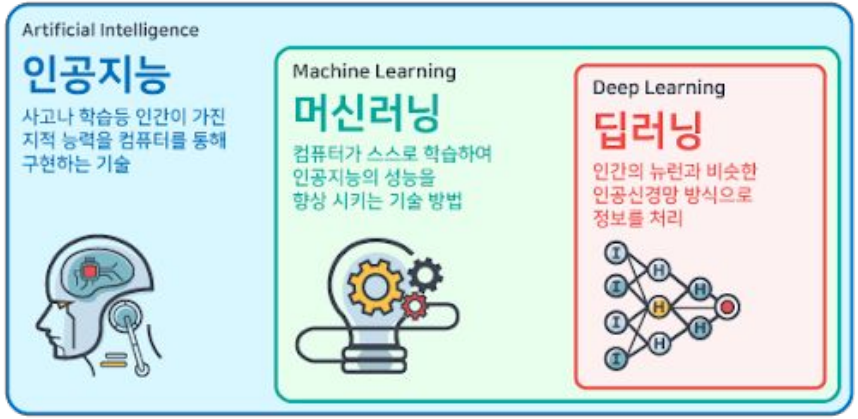
머신 러닝

개념

- 기계가 명시적 프로그래밍 없이, 데이터로부터 스스로 학습하는 기술
- 사람이 일일이 규칙을 입력하지 않아도 데이터 속 패턴을 학습하여 예측·분류·판단을 수행
- 인공지능(AI)의 핵심 하위 분야로, 현재 대부분의 AI 서비스의 기반이 되는 기술

학습 방식

- **지도학습 (Supervised)** - 정답(Label)이 있는 데이터로 학습
- **비지도학습 (Unsupervised)** - 정답 없이 데이터 구조 학습
- **강화학습 (Reinforcement)** - 보상을 통해 행동 최적화



대표 알고리즘

- Decision Tree
- Support Vector Machine
- K-Nearest Neighbors, KNN
- Naive Bayes Classifier

의의

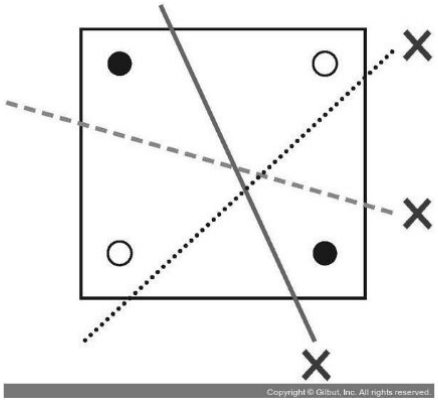
- 데이터 중심 AI 패러다임의 시작
- 이후 딥러닝(Deep Learning)으로 발전하며 음성, 이미지, 텍스트 등 복잡한 데이터 처리 가능

힌튼(Hinton)의 연구

제프리 힌튼(Geoffrey Hinton)

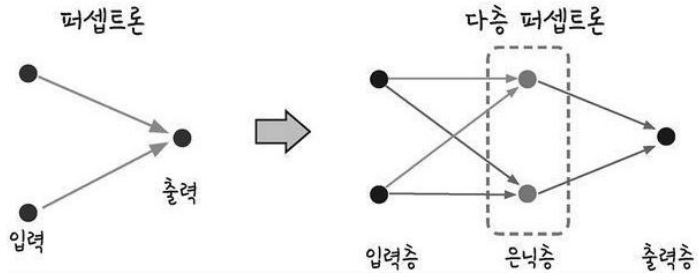
- 1986년 Backpropagation 재발견 및 학습 방법 정립
- 2006년 Deep Belief Network (DBN) 제안 → 다층 신경망의 부활
- 2012년 GPU·데이터 발전과 함께 딥러닝 붐이 폭발 (AlexNet)
- 2018년 튜링상 수상
- 2024년 노벨 물리학상 수상

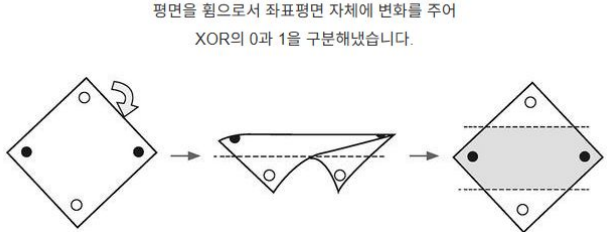




의의

- 힌튼의 연구는 머신러닝의 '수작업 특징 추출' 한계를 해결
- 다층 신경망이 자동으로 '표현'을 학습 (Representation Learning)
- 딥러닝(CNN, RNN, Transformer)의 토대 마련 - 딥러닝의 아버지





Backpropagation XOR 문제 해결

딥러닝

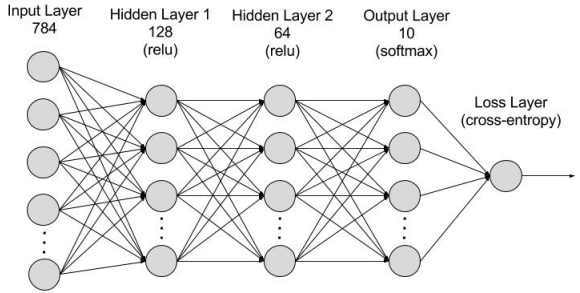
개념

- “데이터의 의미적 표현을 스스로 학습하는 다층 신경망 기반의 인공지능 기술”
  — Geoffrey Hinton (2006, *Deep Belief Network*)
- 머신러닝의 한 분야로, 다수의 은닉층(Hidden Layers) 을 통해 데이터 속 복잡한 패턴과 의미(Representation) 를 자동 학습
- 이미지, 음성, 텍스트 등 비정형 데이터 처리에 탁월

주요 특징

- 2012년 AlexNet 이후 AI 혁명 본격화
- Representation Learning: 데이터의 특징을 스스로 학습
- End-to-End 학습: 입력→출력 전체를 통합적으로 최적화
- 대규모 데이터 + GPU 연산력으로 성능 급상승

유형	모델	대표 응용
이미지	CNN	얼굴 인식, 자율주행
시퀀스	***RNN / LSTM***	***음성 인식, 번역***
생성	GAN	이미지 생성, 딥페이크

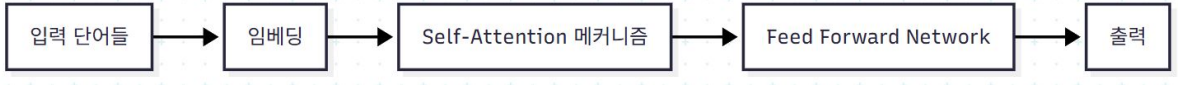


RNN 구조

Transformer (2017, Vaswani et al.)

개념

- 순차적 처리 없이 전체 문맥을 한 번에 학습하는 딥러닝 모델 구조
- Self-Attention 메커니즘을 통해 문장 내 모든 단어 간의 관계(Context) 를 병렬로 학습



핵심 구성 요소

- Self-Attention: 단어 간 관계(문맥) 학습
- Positional Encoding: 단어의 순서 정보 보존
- Multi-Head Attention : 여러 관점에서 문맥을 병렬 학습
- Encoder–Decoder 구조: 입력 이해 → 출력 생성

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

문장 전체를 한 번에 본다

예) The cat sat on the mat

RNN은 단어를 순서대로 봤지만,
Transformer는 모든 단어의 관계를 동시에 고려.
→ 긴 문맥도 이해 가능, 병렬 처리로 학습 속도 급상승.

의의

- 기존 RNN/LSTM의 순차적 한계(Sequential Limitation) 를 극복
- RNN 대비 병렬 학습 가능 → 속도 향상
- 문맥 이해 능력 강화 → 번역, 요약, 질의응답 등 자연어 처리 혁신
- 이후 GPT, BERT, Claude, Gemini 등 LLM의 기본 구조로 자리잡음

대규모 언어 모델 (LLM, Large Language Model)

개념

- 딥러닝 기반의 초대규모 언어 모델
- 대규모 텍스트 데이터를 학습하여 언어의 패턴과 의미를 예측·생성
- 입력된 문맥(context) 을 바탕으로 다음 단어를 예측
- 방대한 데이터와 수십억 개의 파라미터(parameters) 를 학습하여 자연어 이해(NLU) 와 생성(NLG) 을 모두 수행

핵심 기술 요소

- **Transformer**: Attention 메커니즘으로 문맥 관계 학습
- **Token / Embedding**: 단어를 수치화해 의미 공간에 매핑
- **Parameter**: 언어 패턴을 내재화한 학습된 가중치
- **Fine-tuning / Instruction-tuning**: 특정 과업에 맞게 모델을 재조정

의의

- 언어 이해·요약·번역·코드 작성 등 범용 지능의 기초 모델
- 기존의 **규칙 기반 AI**에서 **문맥 기반 AI**로의 패러다임 전환

$P(w_t|w_{1:t-1})$ → 가장 가능성 높은 단어를 생성



LLM 동작 절차

LLM 이해

LLM은 “언어를 이해하는 기계”가 아니라 “언어 패턴을 예측하는 기계”.

- 수십억 문장에서 문맥 확률을 학습
- 언어의 통계적 구조를 예측
- 대규모 데이터 + GPU 학습으로 언어 이해 수준 도달

$P(w_t|w_{1:t-1})$ → 가장 가능성 높은 단어를 생성



LLM 동작 절차

LLM이 ‘생각하는 것처럼’ 보이는 이유

- 실제로는 생각하지 않는다. 패턴 예측이 너무 정교해서 생각처럼 보일 뿐
- 다음에 올 단어를 예측
- 방대한 언어 패턴을 학습한 결과 인간처럼 보임
- LLM은는 통계적 예측 엔진일 뿐, 의식 있는 존재가 아니다.

다음 단어 예측 게임

Q: “오늘 날씨가 너무…”

→ A: “좋아요”

LLM 엔진 비교

- **ChatGPT** → 창의적 생산성과 개발용 API에 강함
  - 창의적 생산성과 개발용 API에 강함
- **Claude** → 안전하고 논리적인 대화형 AI
  - 안전하고 논리적인 대화형 AI
- **Gemini** → 검색·영상·멀티모달 융합형 AI
  - 검색·영상·멀티모달 융합형 AI로 웹서비스 제작 과정에서 강점

엔진	개발사	주요 특징	강점 영역
ChatGPT (GPT-5)	OpenAI	코딩, 디자인 출력 강화, 더 나은 추론·정확성 증가	범용 언어 생성 개발자 생태계
Claude 4 (Opus 4.1, Sonnet 4.5)	Anthropic	“하이브리드 리즈닝” 기능, 장기 과제/코딩에 특화	논리적 추론 안전성 장문 처리
Gemini 2.5 Pro	Google DeepMind	텍스트·이미지·오디오·비디오 멀티모달 지원, 대용량 문맥 처리	멀티미디어 통합, 검색·코딩·융합 서비스

웹 서비스 제작 방식의 변화

과거	현재	미래
개발자가 모든 로직을 직접 작성	AI가 코드, 문서, UI를 자동 생성	개발자는 “프롬프트 디자이너”가 됨
정적 콘텐츠 중심	대화형·지능형 서비스 중심	자율적 에이전트 기반 서비스
사용자 입력 → 결과 출력	사용자 대화 → 맥락 유지 → 개인화 결과	개인 맞춤형 지속 대화형 UX

LLM이 바꾸는 개발 패러다임

- **Augmented Coding** → 개발 과정에 AI 동반
- **AI API Integration** → OpenAI, Gemini, Claude 등 엔진을 백엔드로 연결
- **Autonomous Agent** → AI가 데이터 수집·요약·업데이트를 자동 수행
- **Prompt Engineering** → UI/UX보다 “AI 행동 설계”가 핵심이 됨

영역	변화 방향
개발(Dev)	“코딩”에서 “설계 + 프롬프트”로 전환
디자인(UI/UX)	사용자가 AI와 상호작용하는 흐름 중심
서비스 운영	실시간 데이터·AI 모델 자동 업데이트
콘텐츠 생성	Text → Image → Video 통합 자동화
사용자 경험	AI가 개인 맞춤형 도우미로 진화

웹 서비스 제작의 미래

웹 서비스 개발을 위한 AI 활용 노하우

Prompt Thinking

AI를 활용해 기획·설계·개발 효율을 극대화하기 위한 사고 방식

Prompt Engineering	명확한 목적, 역할, 조건을 가진 명령 설계 기술
Chain of Thought (CoT)	단계별 사고를 유도하여 논리적 출력 확보
Few-shot / Zero-shot Learning	예시를 통한 모델 학습 맥락 제공
Role Prompting	“너는 ~전문가다”로 역할 지정해 맥락 조정
System Prompt Design	모델 전체 톤·스타일을 결정하는 최상위 설정
Prompt Debugging	출력 결과를 기반으로 입력 구조를 개선하는 과정

프롬프트 엔지니어링(Prompt Engineering)

AI에게 명확하고 구조적으로 의도를 전달해 원하는 결과를 이끌어내는 기술

- LLM은 인간의 언어를 명령문(prompt) 형태로 이해
- 따라서 무엇을, 어떤 방식으로, 어떤 톤으로 요청하느냐가 결과 품질을 결정
- 개발·기획 단계 모두에서 AI 활용 능력을 좌우하는 핵심 역량

좋은 프롬프트의 원칙

- **명확하게(Make it explicit)** — 모호한 단어 대신 구체적 표현
- **단계적으로(Decompose the task)** — 복잡한 문제는 단계로 나누기
- **역할 지정(Role prompting)** — 모델의 시점을 설정
- **피드백 루프(Iterate)** — 결과를 기반으로 지속 개선

요소	설명	예시
목표(Goal)	무엇을 생성할지 명확히	“서비스 기획 요약”
역할(Role)	모델에게 역할 부여	“너는 UX 기획 전문가다”
맥락(Context)	배경 정보 제공	“우리 앱은 여행 추천 서비스다”
형식(Format)	출력 형태 지정	“표로 정리해줘 / JSON으로 출력해줘”
제약조건(Constraints)	길이, 톤, 언어 등 제한	“200자 이내, 한국어, 중립적 어조”

사고의 사슬(Chain of Thought)

복잡한 문제를 해결하기 위해, LLM이 스스로 단계별로 사고 과정을 전개하도록 유도하는 프롬프트 기법

- 단순히 정답만 생성하는 것이 아니라 생각의 과정을 먼저 표현하도록 지시
- 모델이 추론 과정을 내부적으로 전개함으로써 논리적 일관성과 문제 해결 품질을 크게 향상시킴

예시

- **일반 프롬프트**

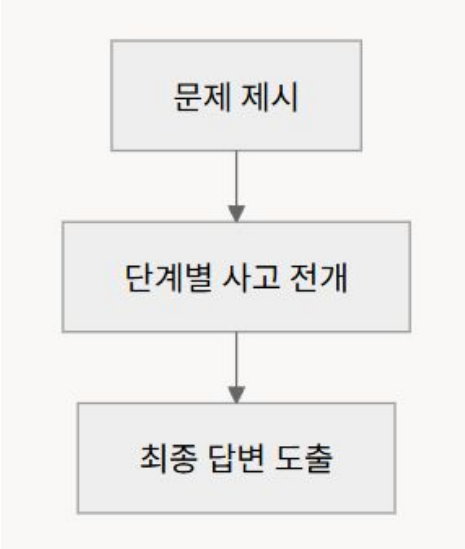
  “3명의 학생이 12개의 사탕을 나누면 몇 개씩 가지게 되나요?”

  → “4개” (정답만 출력)

- **CoT 프롬프트**

  “단계별로 생각해보자. 총 12개의 사탕을 3명에게 똑같이 나누면, 각 학생이 몇 개를 가지는가?”

  → “12 ÷ 3 = 4 → 따라서 각 학생은 4개를 가진다.”



효과

- 논리 강화: 중간 사고 과정을 통해 오답 감소
- 투명성 향상: 모델의 추론 흐름을 관찰 가능
- 정확성 상승: 수학, 코딩, 계획 수립 등 복잡 문제에 유효

Few-shot / Zero-shot Learning

모델이 학습되지 않은 새로운 과제를, 예시의 유무에 따라 이해하고 수행하는 능력

- LLM은 사전학습을 통해 일반 지식을 내재화함
- 이후 프롬프트 안에 예시를 제시하느냐(Few-shot) 또는 없이 바로 요청하느냐(Zero-shot) 에 따라 응답 품질이 달라짐

예시

- **Zero-shot**

  “이 문장을 한 줄로 요약해줘.

  문장: ‘오늘 학교 식당에서 김치볶음밥을 먹었는데 맛있었다.’”

  → “오늘 학교 식당에서 김치볶음밥을 먹었다.” ***(그냥 정보 요약)***

- **Few-shot**

  “아래 예시처럼 ‘감정이 담긴 한 줄 후기’로 요약해줘.

  예시: ‘버스가 늦게 와서 짜증났다.’ → ‘지각해서 기분이 나빴다.’

  문장: ‘오늘 학교 식당에서 김치볶음밥을 먹었는데 맛있었다.’”

  → “점심이 만족스러워 기분이 좋았다.” ***(패턴+감정 표현 학습)***

구분	Zero-shot	Few-shot
예시 제공 여부	없음	있음
프롬프트 구성	“질문만 제시”	“예시 + 질문 제시”
장점	단순, 빠름, 유연함	높은 정확도, 스타일 유지
활용 상황	일반 지식 기반 요청	특정 형식·패턴 유지 필요 시

Few-shot / Zero-shot 학습 비교

활용

- Few-shot은 데이터셋이 적을 때 강력한 “즉석 학습” 도구
- Prompt 안에 입출력 예시(I/O pair) 를 제시하면 모델이 패턴과 형식을 스스로 일반화
- CoT(Chain of Thought)와 결합 시 고난도 문제 해결력 극대화

역할 프롬프팅(Role Prompting)

AI에게 특정 역할(Role)을 부여하여, 응답의 시점·어조·전문성을 조정하는 프롬프트 기법

- LLM은 '누구의 관점으로 말해야 하는가?' 를 지정받을 때 답변의 내용, 깊이, 표현 방식이 달라짐
- 단순 요청보다 맥락이 뚜렷하고 일관된 출력을 유도할 수 있음

예시

- "너는 UX 기획 전문가야.
  아래 서비스 설명을 읽고, 사용자 입장에서 개선 아이디어 3가지를 제시해줘."

효과

- **전문성 강화:** 도메인 지식을 가진 시점으로 답변
- **톤·스타일 제어:** 교수 / 개발자 / 마케터 등 말투 차이 조절
- **일관성 유지:** 긴 대화에서도 동일한 관점 유지
- **팀 협업형 활용:** "AI 팀원"처럼 역할 분담 가능

구분	프롬프트	결과 특징
Non-Role	"이 앱을 평가해줘."	단순한 피드백 중심
Role	"너는 UX 디자이너야. 사용성 기준으로 평가해줘."	구체적, 전문적, 사용자 중심의 답변

시스템 프롬프트 설계(System Prompt Design)

AI의 전반적인 성격, 말투, 목표, 역할 범위를 설계하는 상위 지시문

- 일반 프롬프트는 “질문 단위 지시”
- 시스템 프롬프트는 모델의 행동 원칙과 세계관을 설정하는 기본 세팅값
- LLM이 장시간 일관된 태도와 전문성을 유지하도록 만드는 핵심 기술

구조 및 역할

- System Prompt → AI의 전반적 성격·규칙 정의
- User Prompt → 실제 요청 입력
- Assistant Role → System 규칙을 기반으로 응답

요소	설명	예시
역할(Role)	AI의 정체성 설정	“너는 UX 디자이너다.”
목표(Objective)	대화 목적 정의	“항상 사용자 문제를 해결하는 데 집중해라.”
스타일(Style)	말투·톤 지정	“친근하고 간결하게 말하라.”
출력 형식(Format)	일관된 구조 유지	“모든 답변을 표로 출력하라.”
제약조건(Constraints)	금지·주의 규칙	“사실에 근거하지 않은 내용은 말하지 마라.”

System Prompt Design 요소

예시

- System: “너는 창의적인 서비스 기획자이며, 답변은 항상 표 형식으로 제공한다.”
- User: “새로운 여행 앱 아이디어를 제시해줘.”
- AI: 표로 정리된 여행 앱 아이디어 출력

효과

- 장기 대화에서도 일관된 태도 유지
- 팀용 챗봇·도메인 특화 모델에 브랜드 톤 부여
- 특정 업무용 Custom GPT / Enterprise Agent 설계의 기반

프롬프트 디버깅(Prompt Debugging)

LLM이 예상과 다른 출력을 낼 때, 원인을 분석하고 프롬프트를 수정·개선하는 과정

- AI도 프롬프트가 모호하거나 구조가 불완전하면 오류를 낸다.
- “좋은 프롬프트는 한 번에 완성되지 않는다.” → 디버깅 루프(Feedback Loop) 가 필수
- 개발자의 프롬프트 품질 개선 능력 = AI 활용 역량

개선 예시

- **Before**

  “요약해줘.” → 너무 길고 불필요한 내용 포함

- **After**

  “너는 편집자야. 아래 글을 3문장으로, 200자 이내로 요약해줘.

  핵심 메시지 위주로.”

  → 명확하고 일관된 요약 결과 출력

핵심 포인트

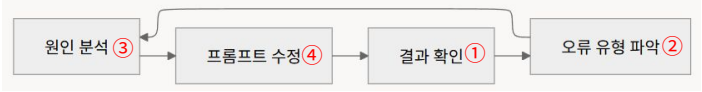
- 모호함을 제거하라
- 출력 형식과 제약을 구체화하라
- 프롬프트 수정은 실험의 반복이다

유형	원인	해결 방법
과도한 출력	범위 미지정	“200자 이내로” 등 제한 추가
반복 응답	맥락 혼선	이전 지시 초기화 후 재요청
불완전한 형식	출력 예시 없음	예시 포맷(Few-shot) 제시
이해 오류	용어 모호	구체적 설명 추가 (“여기서 UX는 사용자 경험을 의미”)

주로 발생하는 오류 유형

단계	설명	예시
① 결과 확인	예상과 다른 출력 검토	“답이 너무 길다”
② 오류 유형 파악	모호함 / 누락 / 형식 불일치	“출력 형식이 지켜지지 않음”
③ 원인 분석	프롬프트 내 지시 불명확	“요약 길이 제한이 없음”
④ 개선 및 재시도	명확히 수정 후 재입력	“200자 이내로 요약해줘.”

Prompt Debugging 절차



웹서비스 개발을 위한유용한 AI 관련 도구들

Task Master - AI 기반 개발 업무 자동화 도구

Claude·ChatGPT·Gemini 등 LLM을 연동해 PRD로부터 개발 작업을 자동으로 생성·관리하는 AI 작업 매니저

- Cursor / VS Code / Windsurf 등 주요 IDE와 통합되어 AI가 프로젝트를 이해하고, 다음 할 일(Task)을 안내

특징

구분	설명
AI 통합 관리	Anthropic(Claude), OpenAI, Google Gemini 등 다양한 모델을 선택·교체 가능
PRD 기반 태스크 자동 생성	/scripts/prd.txt 파일을 분석해 세부 개발 작업 자동 생성
MCP (Model Control Protocol)	Cursor·VSCode에서 직접 Task Master 실행 (npx 연동)
세 가지 모델 구조	Main / Research / Fallback 모델 구성으로 안정적 결과
대화형 명령 지원	“다음 작업이 뭐야?”, “3번 태스크 구현 도와줘” 등 자연어 명령 처리
CLI 명령어	task-master init, parse-prd, list, next, generate 등 명령 기반 제어 가능

활용 시나리오

- 개발팀: PRD 기반 자동 일정 생성 및 관리
- 개인 개발자: AI와 협업하는 프로젝트 플래너
- 교육: LLM 기반 개발 자동화 학습용 도구

https://github.com/eighty9nine/task-master/

React Hook Form

Form의 유효성을 검증하고, 불필요하게 화면이 계속 새로 그려지는 걸 막아주는 성능 향상 도구

문제

- React에서 <input> 값을 관리하려면 useState 사용 시
  - 사용자가 글자를 칠 때마다 setState()가 실행되어 컴포넌트 전체를 다시 그림
  - 입력 필드가 많을수록 느려짐

React Hook Form 사용 시

- 각 입력값을 브라우저 DOM 자체에 저장
- 필요한 순간(검증·제출 등)에만 React에 알려줌
- 불필요한 리렌더링이 거의 없음 = 빠름·가벼움

항목	Virtual DOM (React 기본)	React Hook Form
렌더링 시점	값이 바뀔 때마다 setState() → 리렌더	값 변경 시 리렌더 없음 (DOM이 직접 저장)
속도 개선 방식	변경 부분만 다시 그림(diffing)	변경 자체를 최소화 (uncontrolled input)
폼 관리 방식	컨트롤드 컴포넌트 중심	언컨트롤드 컴포넌트 중심
리렌더 발생 횟수	입력할 때마다	제출·검증 시에만
대표 장점	전체 UI 업데이트 효율적	폼 입력/검증 속도 탁월

https://react-hook-form.com/

Tailwind CSS

디자인 시스템을 코드로 표현하는 유틸리티 클래스 기반 프레임워크

- CSS를 직접 작성하지 않고, **클래스 이름으로 빠르고 일관된 UI 스타일링**
- CSS를 외우는 대신 조합한다는 철학에 기반한 프레임워크로, **디자인 속도·일관성·유지보수성**을 크게 향상 시킴

구분	설명
유틸리티 클래스 중심	bg-blue-500, text-center, p-4 등 조합으로 바로 스타일 적용
빠른 개발 속도	CSS 파일 작성 없이 HTML/JSX 내에서 즉시 디자인 구현
일관된 디자인 시스템	tailwind.config.js에서 색상·폰트·간격 등 테마 통일 관리
반응형 디자인 내장	sm:, md:, lg: 접두사로 반응형 스타일 지정
불필요한 코드 제거(Purge)	실제 사용된 클래스만 빌드 → 번들 크기 최소화
생태계 확장성	daisyUI, Flowbite, shadcn/ui 등 UI 컴포넌트 라이브러리와 호환

https://tailwindcss.com/

Shadcn

React + Tailwind CSS 기반의 컴포넌트 라이브러리 템플릿 모음

- Tailwind 스타일이 입혀진 고품질 UI 컴포넌트 코드를 복붙해서 쓸 수 있게 해주는 도구
- 일반 UI 라이브러리(MUI, Chakra)는 컴포넌트를 가져다 쓰는 형태
- Shadcn/UI는 컴포넌트 코드를 내 프로젝트에 복사해 쓰는 형태
    - 코드처럼 수정·확장 가능

항목	설명
기반 기술	React, Tailwind CSS, Radix UI
특징	라이브러리가 아니라 복사 가능한 소스 코드를 제공 (직접 프로젝트에 포함)
장점	디자인 통일성, 커스터마이징 용이, TypeScript 친화적
단점	자동 업데이트 없음 (직접 코드 관리 필요)
대표 컴포넌트	Button, Input, Card, Modal, Tabs, Dropdown, Toast 등

https://ui.shadcn.com/

개인 프로젝트 평가 가이드라인

2025.11.9 기획서 제출(5)

- 웹 서비스 기획서(PRD: Product Requirement Document), 웹 서비스 원페이저(OnePager)

2025.11.10 기획서 발표(5)

- 웹 서비스 원페이저(OnePager) 발표 5 분 이내

2025.12.1(1차), 2025.12. 8 (2차) 개발 현황 발표(10)

- 발표 10분 이내, 현재 개발중인 기능 소개

2025.12.21 프로젝트 결과 제출 평가(20)

- 서비스 링크, github 저장소, 서비스 소개서(포트폴리오 용)
- ***프로그래밍: (10), 프론트엔드(2), 백엔드(2), Database(2), 기술적 완성도(4) - 다양한 기술의 활용***
- ***기획: (5), 참신성, 사업성, 볼륨***
- ***UX/UI: (5), UX 구성, UI 디자인 완성도***

Q&A